Τι θα δούμε σήμερα

- Πώς η C βλέπει τη μνήμη: διευθύνσεις και δείκτες.
- Κλήση κατά τιμή και προσομοίωση κλήσης κατά αναφορά.
- Η σχέση πινάκων και δεικτών, και οι συμβολοσειρές.
- Στοίβα και σωρός, δυναμική δέσμευση με malloc και free.
- Δομές και μια πρώτη συνδεδεμένη λίστα.
- Τα λάθη μνήμης που συναντούμε πιο συχνά.

Η μνήμη ως πίνακας από bytes

- Κάθε byte έχει μια μοναδική διεύθυνση.
- Μια μεταβλητή καταλαμβάνει διαδοχικά bytes: ένας int συνήθως 4.
- Ο δείκτης είναι μεταβλητή που αποθηκεύει μια διεύθυνση.



Ο p περιέχει τη διεύθυνση του x, δηλαδή «δείχνει» στο x.

Δήλωση και χρήση δείκτη

- &x: η διεύθυνση της μεταβλητής x. *p: η τιμή στη θέση όπου δείχνει ο p (αποαναφορά).
- Ο τύπος του δείκτη λέει πόσα bytes διαβάζονται και πώς ερμηνεύονται.

```
int x = 42;
int *p = &x;      // p holds the address of x
*p = 50;          // now x == 50
printf("%d %d\n", x, *p);   // prints: 50 50
int *q = NULL;    // points nowhere: never dereference it
```

Κλήση κατά τιμή: γιατί η swap αποτυγχάνει

- Η C περνά πάντα αντίγραφα των ορισμάτων στη συνάρτηση.
- Η swap_bad αλλάζει τα αντίγραφά της· οι μεταβλητές του καλούντα μένουν ίδιες.
- Περνώντας διευθύνσεις, η συνάρτηση μπορεί να αλλάξει τα πρωτότυπα.

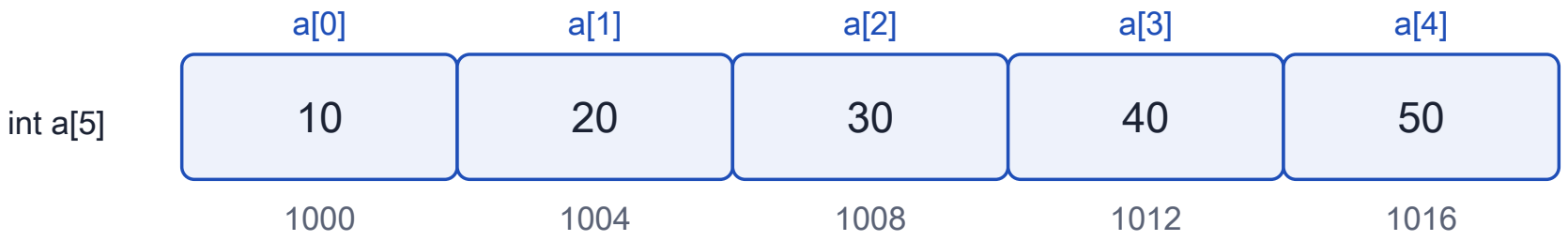
```
void swap_bad(int a, int b) { int t = a; a = b; b = t; }

void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int x = 1, y = 2;
swap_bad(x, y);   // x == 1, y == 2
swap(&x, &y);     // x == 2, y == 1
```

Πίνακες και αριθμητική δεικτών

- Το όνομα ενός πίνακα, στις περισσότερες εκφράσεις, μετατρέπεται σε δείκτη στο πρώτο στοιχείο.
- Ισχύει a[i] == *(a + i). Το a + 1 προχωρά κατά sizeof(int) bytes, όχι κατά 1 byte.
- Όταν περνάμε πίνακα σε συνάρτηση, περνάμε δείκτη: το μέγεθος πρέπει να δοθεί χωριστά.



Συμβολοσειρές στη C

- Μια συμβολοσειρά είναι πίνακας char που τελειώνει με τον χαρακτήρα '\0'.
- Το strlen μετρά χαρακτήρες μέχρι το '\0', χωρίς να το περιλαμβάνει, άρα χρειάζεται βρόχο: O(n).
- Το strcpy δεν ελέγχει το μέγεθος του προορισμού: κίνδυνος υπερχείλισης (buffer overflow).

```
char s[6] = "hello";          // 5 letters + '\0'
char d[4];
strcpy(d, s);                  // WRONG: writes 6 bytes into 4
snprintf(d, sizeof d, "%s", s); // safe: d becomes "hel"
```

Στοίβα και σωρός

- Οι τοπικές μεταβλητές ζουν στη στοίβα και χάνονται όταν τελειώσει η συνάρτηση.

	Στοίβα (stack)	Σωρός (heap)
Ποιος δεσμεύει	Αυτόματα, με την κλήση συνάρτησης	Ο προγραμματιστής, με malloc
Ποιος αποδεσμεύει	Αυτόματα, με την επιστροφή	Ο προγραμματιστής, με free
Μέγεθος	Μικρό και περιορισμένο	Πολύ μεγαλύτερο
Ταχύτητα	Πολύ γρήγορη	Πιο αργή
Διάρκεια ζωής	Όσο τρέχει η συνάρτηση	Μέχρι το free

Δυναμική δέσμευση: malloc και free

- Το malloc(n) επιστρέφει δείκτη σε n bytes ή NULL αν αποτύχει: πάντα ελέγχουμε το αποτέλεσμα.
- Κάθε επιτυχημένο malloc αντιστοιχεί σε ακριβώς ένα free.
- Το calloc μηδενίζει τη μνήμη, ενώ το realloc αλλάζει το μέγεθος ενός τμήματος.

```
int *a = malloc(n * sizeof *a);
if (a == NULL) { return 1; }
for (int i = 0; i < n; i++) a[i] = i * i;
free(a);
a = NULL;   // avoid using it by mistake
```

Συνηθισμένα λάθη μνήμης

Λάθος	Τι σημαίνει	Συνέπεια
Διαρροή μνήμης	Χάνουμε τον δείκτη χωρίς free	Η μνήμη δεν επιστρέφεται όσο τρέχει το πρόγραμμα
Κρεμάμενος δείκτης	Χρήση δείκτη μετά το free	Απροσδιόριστη συμπεριφορά
Διπλό free	free δύο φορές στον ίδιο δείκτη	Απροσδιόριστη συμπεριφορά
Αποαναφορά NULL	*p όταν p == NULL	Συνήθως segmentation fault
Εκτός ορίων	a[n] σε πίνακα n θέσεων	Απροσδιόριστη συμπεριφορά

Εργαλεία όπως το Valgrind και το AddressSanitizer εντοπίζουν αυτά τα λάθη κατά την εκτέλεση.

Δομές (struct)

- Το struct ομαδοποιεί πεδία διαφορετικών τύπων σε έναν νέο τύπο.
- Με μεταβλητή γράφουμε s.age. Με δείκτη γράφουμε p->age, που ισοδυναμεί με (*p).age.
- Περνάμε μεγάλες δομές με δείκτη, για να αποφύγουμε την αντιγραφή τους.

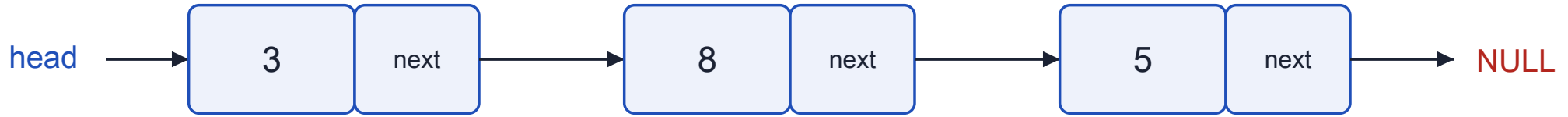
```
struct student { char name[32]; int age; };

void birthday(struct student *p) { p->age++; }

struct student s = { "Maria", 19 };
birthday(&s);     // s.age == 20
```

Συνδεδεμένη λίστα

- Κάθε κόμβος έχει μια τιμή και δείκτη στον επόμενο. Εισαγωγή στην αρχή σε O(1), αναζήτηση σε O(n).




```
struct node { int value; struct node *next; };

struct node *push(struct node *head, int v) {
    struct node *n = malloc(sizeof *n);
    n->value = v;  n->next = head;
    return n;      // n is the new head
}
```

Ανακεφαλαίωση

- Ο δείκτης αποθηκεύει διεύθυνση: & για να την πάρουμε, * για να φτάσουμε στην τιμή.
- Η C περνά ορίσματα κατά τιμή· για να αλλάξει μια συνάρτηση μια μεταβλητή, της δίνουμε τη διεύθυνσή της.
- a[i] και *(a + i) είναι το ίδιο πράγμα.
- Ό,τι δεσμεύουμε με malloc το αποδεσμεύουμε με free, ακριβώς μία φορά.
- Η μνήμη στη στοίβα χάνεται με την επιστροφή: ποτέ δεν επιστρέφουμε δείκτη σε τοπική μεταβλητή.